Тема 17: Модели на разпределени софтуерни архитектури. Среди и протоколи.

ТЕОРИЯ

1. Паралелна и разпределена обработка — параметри и метрика

Основни понятия

Паралелна обработка = обработка от няколко **процеси/нишки (работници)** едновременно. Цел: **ускорение**.

Разпределена обработка = паралелна + процесите са на **различни машини (изчислителни възли)**. Цел: ускорение + **функционално разслояване**.

p = брой работници (паралелизъм)

Два вида паралелизъм:

- **Хардуерен** — брой ядра/машини (физически)
- **Софтуерен** — брой нишки/процеси (логически)

  Ускорението от softw. паралелизъм е ограничено от hardware-а: 8 нишки ≠ 8× при < 8 ядра.

Грануларност

Грануларност = степен на декомпозиция (на колко подзадачи се разбива задачата):

Вид	**Подзадачи**	**Разпределение**	**Балансиране**
Едра	Малко	Бързо	Лошо
Фина	Много	Бавно	Добро
Средна	Компромис	Компромис	Компромис ← желана

Балансиране

Балансиране = всеки работник получава **равен брой задачи** по очакване.

> Времето на паралелния алгоритъм = времето на **най-бавния** работник → нежелателно един да приключи много по-рано.

По времето на осъществяване:

- Статично — разпределението е определено преди изпълнение
- Динамично — разпределението се адаптира по хода

По структурата:

- Централизирано (топология звезда)
- Разпределено (верига, решетка, хиперкуб)
- Йерархично (клъстери — дърво)

Синхронизация

Вид	**Описание**
Асинхронни	Не обменят данни; процесите не знаят за съществуването на другите
Локално синхронни	Всеки работник комуникира само с дефинирани съседи
Глобално синхронни	Всеки работник може да комуникира с всеки (пълен граф)

Тип декомпозиция

Тип	**Означение**	**Описание**
По данни	SPMD (Single Program, Multiple Data)	Една програма върху различни данни
По управление	MPMD (Multiple Program, Multiple Data)	Различни програми, различни данни

Метрики

Ускорение (Speedup): $$S_p = \frac{T_1}{T_p}$$

- T_1 = последователно изпълнение (1 работник)
- T_p = паралелно изпълнение (p работника)
- Обикновено: $1 < S_p < p$

Ефикасност (Efficiency): $$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p}$$

- Нормализирано ускорение: стойности обикн. в (0, 1); удобно в %
- $E_p = 1$ (100%) = перфектно линейно ускорение (теоретичен максимум)

Цена (Cost): $$C_p = p \cdot T_p$$

- Идеална цена = T_1 (= последователното) → $C_p \approx T_1$

Аномалии:

Аномалия	Условие	Причина
Суперлинейна	$S_p > p$	Данните не се побират в RAM → последователно е бавно заради I/O
Нелинейна (регресия)	$S(p+1) < S_p$	Комуникационни overhead-и надхвърлят ускорението

Закон на Амдал (Amdahl's Law): $$S_p = \frac{1}{(1-f) + f/p}$$

- f = дял от паралелизирания код
- Ако f=0.9, дори $p \to \infty$: $S_p \leq 10$

---

2. Модели на разпределените софтуерни архитектури

2.1 Процедурни модели

- Базирани на **последователно извикване на процедури**
- Разпределени чрез **RPC (Remote Procedure Call)** или **RMI (Remote Method Invocation)**
- Извикване на процедура на друг възел изглежда като **локална** за извикващия
- Прозрачно за разработчика

2.2 Обектни модели

- Базирани на **ООП принципи**: абстракция, композиция, наследяване, капсулация, полиморфизъм
- Независими, повторно-използваеми обекти си сътрудничат
- Пример: CORBA, EJB

2.3 Потокови модели

Системата = последователност от **трансформации на данни**.

Подвид	Описание	Изпълнение
Batch sequential	Модули един след друг; изходът = вход на следващ	Отложено
Pipe and Filter	Паралелизъм (конвейеризация); всеки филтър обработва поток	Интерактивно
Process Control	Pipe and Filter + real-time ограничения	Реално време

2.4 Контекстни (Data-Centric) модели

- Централизирано **хранилище** за данни
- Всички компоненти достъпват хранилището

Подвид	**Инициатива**	**Описание**
Repository	Активни агенти	Хранилище = СУБД/CORBA/Web услуги; агентите правят заявки
Blackboard	Хранилището	Агентите са абонати за events при промяна на данните (реактивно)

2.5 Йерархични модели

Подвид	**Описание**
Слоест модел (Layered)	Слоеве; всеки предоставя услуги на по-горния. Пример: OSI, TCP/IP стек, Unix
Master/Slaves	Главна програма разпределя работа на подчинени

2.6 Асинхронни модели

Комуникация чрез **message passing** (имплицитни асинхронни обръщения).

Подвид	**Описание**	**Разновидности**
Producer/Consumer	Обмен 1:1	Интерактивен (online) / Отложен (offline)
Publish/Subscriber	Обмен 1:много	Topic-based / Context-based

2.7 Интерактивни модели

Интензивен потребителски интерфейс. Декомпозиция на 3 модула:

- **Представяне (View)** — UI, визуализация на изходни данни
- **Данни (Model)** — данновия модел, базова функционалност
- **Управление (Controller)** — комуникации, управление на процеси, конфигурация

Два вида:

- **MVC** (Model-View-Controller) — web приложения, GUI frameworks
- **PAC** (Presentation-Abstraction-Control) — за сложни йерархични UI системи

---

3. Организация на разпределените приложения

3.1 Клиент-Сървър

Класически **MPMD модел**.

- **Сървър** = реактивен процес; имплементира приложна логика и данновия модел
- **Клиент** = активен интерфейсен процес; използва услугите на сървъра

Вид клиент	**Описание**
Тънък (thin)	Делегира цялата работа на сървъра; само UI
Дебел (fat)	Изпълнява част от бизнес логиката локално

3.2 Двуслойна архитектура (2-tier)

```
[Клиент: UI + Бизнес логика] ⟷ [Сървър: Данни]
```

Пример: Традиционен файл-сървър; десктоп приложение с база данни.

3.3 Трислойна архитектура (3-tier)

```
[Presentation Layer] ⟷ [Application Layer] ⟷ [Data Layer]
     (Клиент: UI)       (Бизнес логика)         (База данни)
```

- Presentation: UI (браузър, мобилно приложение)
- Application: бизнес логика (сървър на приложения)
- Data: управление на данните (СУБД)

3.4 N-слойна архитектура

Допълнителни **middleware слоеве** между клиента и базата данни.
Всеки слой е клиент на по-долния и сървър на по-горния.

Многослойни сървъри:

- Междинни слоеве (middleware) — между интерфейсния и вътрешния слой
- Вътрешен слой — данни или нискониво комуникации

3.5 Peer-to-Peer (P2P)

Всеки участник е едновременно **клиент и сървър**. Децентрализирано.

Характеристики: децентрализирана топология, самоорганизация, скалируемост, отказоустойчивост, анонимност.

Вид P2P	**Топология**	**Производителност**
Неструктурирано	Произволна (Gnutella)	Непредвидима
Структурирано	Изрично зададена (хорда, DHT)	Гарантирана
Хибридно/йерархично	Ниво 1 структурирано, ниво 2	Добра

Вид P2P	Топология	Производителност
	неструктурирано	

Пример: BitTorrent, Bittorrent trackers.

3.6 Сървъри за приложения и Web-сървъри

Тип	Роля	Примери
Application Server	Бизнес логика; управление на приложни услуги	Apache Tomcat, JBoss, WildFly
Web Server	HTTP/HTTPS; предоставя уеб съдържание	Apache HTTP, Nginx, IIS

3.7 Метасистеми и Грид

Метасистеми = обединяват ресурси от различни мрежи в общо изчислително пространство. Пример: Globus Toolkit

Грид = специализирана метасистема; много компютри работят заедно за сложни задачи, споделяйки ресурси. Пример: SETI@home, LHC Grid.

3.8 Сервизно-ориентирана архитектура (SOA)

- Основни градивни блокове = **услуги** с добре дефинирани интерфейси
- Комуникация чрез стандартни протоколи (SOAP, REST)
- Слабо свързване (loose coupling); повторна употреба
- Пример: Web Services, ESB (Enterprise Service Bus)

Микроуслуги (Microservices) = SOA с по-малки, автономни услуги:

- Всяка услуга = отделен деплоймент
- Независимо мащабиране
- Пример: Docker + Kubernetes

3.9 Моделно-ориентирана архитектура (MDA)

- Разработката се ръководи от **формални модели** (UML)
- Разделяне на независимия от платформата модел (PIM) от специфичния (PSM)
- Код се генерира автоматично от модели

3.10 Аспектно-ориентирана архитектура (AOP)

- Отделяне на cross-cutting concerns (логване, сигурност, транзакции) в отделни **аспекти**
- Аспектите се вплитат автоматично (weaving)
- Пример: Spring AOP

3.11 Софтуерни агенти

Самостоятелни програми, работещи в разпределена среда:

- Комуникират помежду си
- Вземат автономни решения
- Адаптират се към условията
- Пример: интелигентни агенти за търсене, агенти за договаряне

КАК СЕ РЕШАВАТ ЗАДАЧИ

> Изрично в конспекта: „Анализ на изчислителната сложност, ускорението, ефективността и цената на обработка."

Шаблон за решение

Дадено: Алгоритъм с последователно T_1, брой работници p, измерено T_p.

Изчисли:

$$S_p = \frac{T_1}{T_p} \quad \text{(Ускорение)}$$

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \quad \text{(Ефикасност)}$$

$$C_p = p \cdot T_p \quad \text{(Цена)}$$

Примерна задача

Условие: Последователен алгоритъм работи за T_1 = 100 сек. При p = 4 работника: T_p = 30 сек.

Решение:

- S_4 = 100/30 ≈ **3.33** (< 4 → нормално, има комуникационен overhead)
- E_4 = 3.33/4 ≈ **0.83 = 83%** (добра ефикасност)
- C_4 = 4 × 30 = **120 сек** (> T_1=100 → цената е по-висока от последователното)

Интерпретация:

- S_p = 3.33 < p = 4 → нормален случай (комуникационни закъснения)
- E_p = 83% → добро балансиране
- $C_p > T_1$ → има overhead; оптимизацията е на балансиране/грануларност

Типични въпроси

Q: Дадени са T_1 и T_p за различни p — начертай крива на ускорението.

- Изчисли S_p за всяко p, нанеси на графика
- Идеалното е $S_p = p$ (права линия)
- Обичайно: крива, която насища след определено p

Q: При какво p ефикасността пада под 70%?

- Намери p, при което $E_p = S_p/p < 0.7$

Q: Закон на Амдал — максимално ускорение при 80% паралелизируем код. $$S_{max} = \frac{1}{1 - f} = \frac{1}{0.2} = 5$$

Бърза справка

Формула	**Значение**
$S_p = T_1/T_p$	Ускорение
$E_p = S_p/p$	Ефикасност (0–1)
$C_p = p \cdot T_p$	Цена (идеална = T_1)
$S_p > p$	Суперлинейна аномалия
$S(p+1) < S_p$	Нелинейна аномалия
f = паралелна дял	Закон на Амдал: $S\infty = 1/(1-f)$
SPMD	Една програма, различни данни
MPMD	Различни програми, различни данни
MVC	Model-View-Controller
SOA	Услуги с дефинирани интерфейси
P2P	Всеки = клиент + сървър